Aula prática: explorando `vitest-mock-extended` no REPL do Node

Bloco 0 — entrar no laboratório

Rode isso

```
node
```

Depois, no REPL:

```
const { mock, mockDeep } = await import('vitest-mock-extended')
```

Observe isso

O import deve funcionar sem erro.

Conclusão

Agora você consegue criar doubles tipados por estrutura, em vez de começar apenas com `vi.fn()` solto.

Bloco 1 — entender a ideia central

Rode isso

```
const userRepository = mock()
userRepository
```

Observe isso

Você receberá um objeto-proxy de mock.

Conclusão

Diferente de `vi.fn()`, aqui o ponto de entrada não é “uma função mockada”, mas sim um **objeto mockado**.

A proposta do `vitest-mock-extended` é:

- simular objetos tipados
- criar métodos mockados sob demanda

- facilitar mocking de dependências de serviço, repository, gateway, client etc.

Bloco 2 — criar um mock com shape explícito

Rode isso

```
const userRepository = mock()
```

Depois:

```
userRepository.findById
```

Observe isso

Esse acesso já deve te mostrar algo que se comporta como mock function.

Conclusão

O objeto retornado é “preguiçoso”: ao acessar uma propriedade de método, ele expõe um mock configurável.

Mas o uso realmente interessante aparece quando existe **tipo**.

Bloco 3 — usar com tipo real no REPL

No REPL puro do Node, você não tem TypeScript sendo checado em tempo real como no editor, então vamos simular a ideia mentalmente.

Imagine esta interface:

```
type User = {
  id: string
  name: string
}

type UserRepository = {
  findById(id: string): Promise<User | null>
  save(input: { name: string }): Promise<User>
}
```

No arquivo `.ts`, o uso seria:

```
const userRepository = mock<UserRepository>()
```

Observe isso

No REPL Node puro, esse trecho com generics TS não roda diretamente, porque o REPL está em JavaScript.

Conclusão

Para explorar no REPL, você vai inspecionar comportamento em JavaScript.
Para explorar tipagem, o lugar ideal é um arquivo `.ts` de teste ou playground TypeScript.

Bloco 4 — experimentar um método mockado

Rode isso

```
const repo = mock()
repo.findById.mockResolvedValue({ id: 'u1', name: 'John' })
```

Depois:

```
const result = await repo.findById('123')
result
```

Observe isso

O método `findById` se comporta como uma mock function configurável.

Conclusão

No `vitest-mock-extended`, os métodos do objeto já vêm com API tipo:

- `mockReturnValue`
- `mockResolvedValue`
- `mockImplementation`
- etc.

Ou seja: por baixo, você continua trabalhando com a mecânica de mocks do Vitest.

Bloco 5 — inspecionar o histórico do método

Rode isso

```
repo.findById.mock.calls
repo.findById.mock.results
repo.findById.mock.settledResults
```

Observe isso

Você verá o mesmo tipo de rastreamento que explorou com `vi.fn()` .

Conclusão

A grande diferença não é o formato do mock em si, mas **como ele é criado**:

- `vi.fn()` → você mocka uma função
- `mock()` → você mocka um objeto cujos métodos já são mocks

Bloco 6 — múltiplos métodos no mesmo objeto

Rode isso

```
repo.save.mockResolvedValue({ id: 'u2', name: 'Mary' })
await repo.save({ name: 'Mary' })
```

Depois:

```
repo.findById.mock.calls
repo.save.mock.calls
```

Observe isso

Cada método mantém seu próprio estado de mock, independentemente dos demais.

Conclusão

Esse é um dos ganhos mais fortes para backend: você monta doubles de dependências com vários métodos sem precisar declarar `vi.fn()` manualmente para cada um.

Bloco 7 — mudar comportamento com `mockImplementation`

Rode isso

```
repo.findById.mockImplementation(async (id) => {
  return { id, name: `user:${id}` }
})
```

Depois:

```
await repo.findById('abc')
repo.findById.mock.calls
repo.findById.mock.results
repo.findById.mock.settledResults
```

Observe isso

Agora o retorno depende do argumento passado.

Conclusão

A mesma regra continua valendo:

- `mockResolvedValue` para valor fixo assíncrono
- `mockImplementation` quando o retorno depende dos parâmetros

Bloco 8 — sequência com `mockResolvedValueOnce`

Rode isso

```
repo.findById
  .mockResolvedValue({ id: 'default', name: 'Default' })
  .mockResolvedValueOnce({ id: '1', name: 'First' })
  .mockResolvedValueOnce({ id: '2', name: 'Second' })
```

Depois:

```
await repo.findById('x')
await repo.findById('y')
await repo.findById('z')
await repo.findById('w')
```

E então:

```
repo.findById.mock.calls
repo.findById.mock.settledResults
```

Observe isso

Você verá a fila de respostas temporárias seguida do fallback padrão.

Conclusão

Esse padrão é excelente para simular:

- primeira busca retorna valor
- segunda retorna outro valor
- depois volta ao padrão

Bloco 9 — limpar o histórico de um método

Rode isso

```
repo.findById.mockClear()
```

Depois:

```
repo.findById.mock.calls
repo.findById.mock.results
```

Observe isso

O histórico do método fica vazio.

Conclusão

Você limpa o estado do **método específico**, não necessariamente do objeto inteiro.

Bloco 10 — resetar o comportamento do método

Rode isso

```
repo.findById.mockReset()
```

Depois:

```
await repo.findById('123')
repo.findById.mock.results
```

Observe isso

Depois do reset, o método perde a configuração customizada e volta ao comportamento base do mock.

Conclusão

`mockReset()` no método faz com que você recomece a configuração daquele ponto.

Bloco 11 — deep mock: quando a dependência é aninhada

Agora vamos para o que realmente diferencia `mockDeep()` .

Rode isso

```
const db = mockDeep()
```

Depois:

```
db.user.findById.mockResolvedValue({ id: 'u1' })
await db.user.findById('123')
db.user.findById.mock.calls
```

Observe isso

Mesmo sem declarar manualmente `user` , `findById` , etc., o acesso aninhado funciona.

Conclusão

`mockDeep()` cria mocks profundos para estruturas aninhadas.

Esse é o recurso ideal quando você tem algo assim:

- `prisma.user.findUnique`
- `http.client.users.get`
- `broker.publisher.payment.emit`

Bloco 12 — comparar `mock()` e `mockDeep()`

Rode isso

```
const shallow = mock()
const deep = mockDeep()
```

Depois tente mentalmente estas formas:

```
shallow.user.findById
deep.user.findById
```

Observe isso

A diferença conceitual é:

- `mock()` é melhor para objetos cuja estrutura principal você já conhece e acessa diretamente
- `mockDeep()` é melhor para cadeias aninhadas

Conclusão

Regra prática:

- dependência simples com métodos diretos → `mock<T>()`
- dependência rica/aninhada → `mockDeep<T>()`

Bloco 13 — cenário típico de backend

Imagine este contrato:

```
type PaymentGateway = {
  authorize(input: { amount: number }): Promise<{ transactionId: string }>
  cancel(transactionId: string): Promise<void>
}
```

Num teste TypeScript real, você faria:

```
const gateway = mock<PaymentGateway>()

gateway.authorize.mockResolvedValue({ transactionId: 'tx-1' })
gateway.cancel.mockResolvedValue(undefined)
```

Observe isso

Cada método já nasce pronto para configuração e inspeção.

Conclusão

Isso reduz muito o boilerplate de teste de aplicação e casos de uso.

Bloco 14 — por que isso é melhor que montar tudo na mão

Rode isso

Pense nesta abordagem manual com Vitest puro:

```
const repo = {
  findById: vi.fn(),
  save: vi.fn(),
}
```

E compare mentalmente com:

```
const repo = mock<UserRepository>()
```

Observe isso

Com `mock<UserRepository>()` , você ganha:

- intenção mais clara
- shape alinhado ao contrato
- menos código repetitivo
- melhor experiência com autocomplete e tipagem em TS

Conclusão

`vitest-mock-extended` é especialmente valioso quando você trabalha guiado por interfaces e portas.

Bloco 15 — explorar um método como “mock puro”

Rode isso

```
const service = mock()
service.execute.mockReturnValue('ok')
service.execute('x')
service.execute.mock.calls
service.execute.mock.results
```

Observe isso

O método `execute` se comporta exatamente como um mock básico do Vitest.

Conclusão

Mentalmente, você pode pensar assim:

- `mock()` cria um objeto
- cada método desse objeto é, na prática, um mock function

Bloco 16 — sessão guiada curta

Rode isso

```
const repo = mock()

repo.findById.mockResolvedValue({ id: '1', name: 'John' })
await repo.findById('1')

repo.findById.mock.calls
repo.findById.mock.results
repo.findById.mock.settledResults

repo.findById.mockImplementation(async (id) => ({ id, name: `user:${id}` }))
await repo.findById('2')

repo.findById.mock.calls
repo.findById.mock.settledResults

repo.findById.mockClear()
repo.findById.mock.calls
```

Observe isso

Essa sequência te mostra:

- configuração inicial
- chamada
- inspeção de histórico
- troca de implementação
- limpeza de estado

Conclusão

Esse já é o fluxo básico real de uso em testes backend.

Exercício extra: deep mock com cara de Prisma

Rode isso

```
const prisma = mockDeep()

prisma.user.findUnique.mockResolvedValue({ id: 'u1', name: 'John' })
prisma.user.create.mockResolvedValue({ id: 'u2', name: 'Mary' })

await prisma.user.findUnique({ where: { id: 'u1' } })
await prisma.user.create({ data: { name: 'Mary' } })

prisma.user.findUnique.mock.calls
prisma.user.create.mock.calls
```

Observe isso

Você mocka cadeias aninhadas sem declarar manualmente cada nível.

Conclusão

Esse é um dos usos mais naturais de `mockDeep()` em backend Node + TypeScript.

Mapa mental para decorar

`mock()`

Use quando a dependência é algo como:

```
repository.findById()
repository.save()
mailer.send()
gateway.authorize()
```

`mockDeep()`

Use quando a dependência é algo como:

```
prisma.user.findUnique()
client.users.byId.get()
bus.publisher.payment.emit()
```

O que inspecionar sempre

Em qualquer método mockado:

```
repo.findById.mock.calls
repo.findById.mock.results
repo.findById.mock.settledResults
```

e configurar com:

```
repo.findById.mockReturnValue(...)
repo.findById.mockResolvedValue(...)
repo.findById.mockImplementation(...)
repo.findById.mockClear()
repo.findById.mockReset()
```